

Cache-Optimized Numerical Computation Engine

A C++ library implementing high-performance matrix operations by exploiting CPU cache/memory hierarchy, benchmarked against naive implementations.

Course Foundations

This project applies concepts from three courses: **Computer Organization and Architecture** (CPU cache hierarchy, spatial/temporal locality), **Numerical Methods Laboratory** (Gaussian elimination, numerical stability via pivoting), and **Applied Computational Methods** (algorithmic performance analysis).

Problem Statement

Standard (naive) matrix multiplication in the *i-j-k* loop order has poor cache locality: accessing matrix B column-by-column causes a cache miss almost every iteration, since B is stored in row-major order in memory. For large matrices, this makes multiplication far slower than the $O(n^3)$ operation count alone would suggest. This project implements and benchmarks two optimizations against the naive baseline.

Approach

- 1. Loop Reordering (i-k-j):** Reordering the three loops so that both the inner matrix (B) and output matrix (C) are accessed row-wise (stride-1) instead of column-wise, improving spatial locality with zero extra memory cost.
- 2. Blocked (Tiled) Multiplication:** The matrices are partitioned into $BLOCK \times BLOCK$ sub-tiles ($BLOCK = 32$) sized to fit within L1/L2 cache. Multiplication proceeds tile-by-tile, so each tile's data stays resident in cache across the computation, rather than being evicted and re-fetched from main memory repeatedly.
- 3. Gaussian Elimination with Partial Pivoting:** A numerically stable linear system solver ($Ax = b$) is implemented separately, choosing the largest-magnitude pivot in each column to minimize floating-point error accumulation (from Numerical Methods coursework).

Key Code — Blocked Multiplication

```
Matrix blockedMultiply(const Matrix &A, const Matrix &B, int n, int BLOCK) {
    Matrix C(n, vector<double>(n, 0.0));
    for (int ii = 0; ii < n; ii += BLOCK)
        for (int kk = 0; kk < n; kk += BLOCK)
            for (int jj = 0; jj < n; jj += BLOCK) {
                int iMax = min(ii+BLOCK,n), kMax = min(kk+BLOCK,n), jMax = min(jj+BLOCK,n);
                for (int i = ii; i < iMax; i++)
                    for (int k = kk; k < kMax; k++) {
                        double a = A[i][k];
                        for (int j = jj; j < jMax; j++)
                            C[i][j] += a * B[k][j]; // stride-1 on B and C
                    }
            }
    return C;
}
```

Benchmark Results (measured on this machine)

Matrix Size (n)	Naive (ms)	Loop-Reordered (ms)	Blocked (ms)	Speedup (Naive/Blocked)
-----------------	------------	---------------------	--------------	-------------------------

128	2.032	0.956	0.956	2.13x
256	18.796	7.392	8.201	2.29x
512	164.216	63.145	74.555	2.20x

Results were verified for correctness by comparing checksums of naive and blocked outputs (matched within floating-point tolerance). The blocked implementation achieves a consistent **~2.1–2.3x speedup** over the naive baseline across all tested sizes, purely from improved cache utilization — no change in asymptotic complexity (both remain $O(n^3)$).

Gaussian Elimination Verification

Solved the system $Ax = b$ for $A = \begin{bmatrix} 2 & 1 & -1 \\ -3 & -1 & 2 \\ -2 & 1 & 2 \end{bmatrix}$, $b = [8, -11, -3]$. Obtained $x = [2.000, 3.000, -1.000]$, matching the exact analytical solution, confirming correctness of the pivoting-based solver.

Tech Stack

C++17, compiled with g++ -O2 optimization, std::chrono for benchmarking.

Possible Extensions

SIMD vectorization (AVX intrinsics), multi-threaded blocked multiplication (OpenMP), and comparison against BLAS/LAPACK reference implementations for absolute performance context.